Apache Groovy

Ошибки, ввод-вывод, конкурентность и JVM

Часть 3 из 3

Иерархия ошибок JVM

Throwable

Error
серьёзная проблема JVM или среды

Exception
сбой, который можно обработать

Проверяемые
`IOException`

Непроверяемые
`RuntimeException`

Java
Проверяемое исключение требует `catch` или `throws`.

Groovy
`catch` и `throws` не обязательны.

Встроенные исключения

Среда	Исключение	Причина
Java / JVM	`IllegalArgumentException`	неверный аргумент
Java / JVM	`NumberFormatException`	строка не число
Java / JVM	`IOException`	чтение или запись
Groovy	`MissingMethodException`	метод не найден
Groovy	`MissingPropertyException`	свойство не найдено

try, catch и finally

```
int parseAge(String text) {
    Integer.parseInt(text)
}

['21', 'twenty'].each { text ->
    try {
        println "Age: ${parseAge(text)}"
    } catch (NumberFormatException exception) {
        println "Cannot parse '$text': " +
                exception.class.simpleName
    } finally {
        println "Processed: $text"
    }
}
```

try операция с риском ошибки

catch обработка выбранного типа

finally выполняется после обеих веток

```
Age: 21
Processed: 21
Cannot parse 'twenty':
NumberFormatException
Processed: twenty
```

throw и собственный тип

```
class InvalidScoreException
        extends RuntimeException {
    InvalidScoreException(String message) {
        super(message)
    }
}

int validateScore(int score) {
    if (!(score in 0..100)) {
        throw new InvalidScoreException(
            "Score outside 0..100: $score")
    }
    score
}
```

Собственный тип исключения должен наследоваться от `RuntimeException`.

Вывод

```
Valid score: 95
Invalid score:
Score outside 0..100: 120
```

Проверки с assert

assert проверяет инвариант или ожидание теста

Ложное условие создаёт `AssertionError`.

Power Assert показывает выражение и его значения

```
def prices = [10, 15, 20]
def expectedTotal = 50

assert (prices.sum() ==
        expectedTotal):
       'Unexpected total'
```

```
Unexpected total.
Expression:
(prices.sum() == expectedTotal)
Values: expectedTotal = 50
```

Чтение файла

```
Path inputFile =
    Path.of('data/names.txt')

inputFile.eachLine('UTF-8') { line, number ->
    def (name, age) = line.split(',')*.trim()
    println "$number. $name, age $age"
}
```

data/names.txt

```
Alice,18
Bob,21
Carol,20
```

Path путь к файлу

UTF-8 кодировка текста

eachLine построчное чтение

Вывод

```
1. Alice, age 18
2. Bob, age 21
3. Carol, age 20
```

Запись файла

```
Path report = Files.createTempFile(
    'groovy-report-', '.txt')

report.withWriter('UTF-8') { writer ->
    writer.writeLine('language=Groovy')
    writer.writeLine('examples=12')
}

report.eachLine('UTF-8') { line -> println line }
```

Вывод

```
Saved report:
language=Groovy
examples=12
```

Writer символьная запись

withWriter закрывает ресурс после closure

finally удаляет временный файл

Источники ввода

args аргумент запуска

System.console() терминал

System.in pipe или файл

Вывод

```
Source: command-line argument
Hello, Alice!
Source: standard input
Hello, Grace!
```

```
if (args) {
    name = args[0]
    source = 'command-line argument'
} else if (console) {
    name = console.readLine('Name: ')
    source = 'console'
} else {
    name = System.in.newReader('UTF-8')
        .readLine()
    source = 'standard input'
}
```

Конкурентность и параллелизм

Задача
Отдельная работа, которую нужно выполнить.

Поток
Последовательно выполняет инструкции. Потоки процесса разделяют объекты в куче.

Конкурентность
Задачи продвигаются в перекрывающиеся промежутки времени.

Параллелизм
Задачи выполняются одновременно на нескольких ядрах.

Создание и ожидание потока

```
println 'Main: before start'

Thread worker = new Thread({
    println 'Worker: started'
    Thread.sleep(100)
    println 'Worker: finished'
} as Runnable)

worker.start()
worker.join()

println 'Main: after join'
```

Runnable
задача без результата

start()
запускает новый поток

join()
ждёт завершения потока

```
Main: before start
Worker: started
Worker: finished
Main: after join
```

Общее состояние и гонка

Операция value++

```
int current = counter.value
int updated = current + 1
counter.value = updated
```

Один возможный порядок

Поток A	читает 0
Поток B	читает 0
Поток A	записывает 1
Поток B	записывает 1

Общее состояние

Изменяемый объект, доступный нескольким потокам.

Гонка

Результат зависит от взаимного порядка действий потоков. Здесь одно обновление теряется.

Вывод

```
Unsafe counter: 1 (expected 2)
```

synchronized и блокировка

```
class SynchronizedCounter {
    private int value

    synchronized void increment() { value++ }

    synchronized int currentValue() { value }
}

def counter = new SynchronizedCounter()
List<Thread> workers = (1..2).collect {
    new Thread({ 1_000.times {
        counter.increment()
    } } as Runnable)
}
workers*.start()
workers*.join()
```

Критическая секция
Код, который меняет общий счётчик.

Блокировка
Один поток выполняет секцию, остальные ждут.

Видимость
Следующий поток видит изменения после входа.

```
Synchronized counter: 2000
(expected 2000)
```

Пул потоков

.newFixedThreadPool(2)
создаёт два рабочих потока
остальные задачи ждут в очереди

submit()
передаёт задачу свободному потоку

shutdown()
завершает пул после уже отправленных задач

```
def pool = Executors
    .newFixedThreadPool(2)

try {
    // pool.submit(...)
} finally {
    pool.shutdown()
}
```

Результат задачи через Future

```
def numbers = [2, 3, 4, 5]
ExecutorService pool =
    Executors.newFixedThreadPool(2)
try {
    List<Future<Integer>> futures =
        numbers.collect { number ->
        pool.submit({ number * number }
                    as Callable<Integer>)
    }
    def squares = futures.collect { it.get() }
    println "Squares: $squares"
    println "Sum: ${squares.sum()}"
} finally {
    pool.shutdown()
}
```

Callable
задача с результатом

submit()
отправляет задачу в пул

Future<T>
`get()` ждёт и возвращает результат

```
Squares: [4, 9, 16, 25]
Sum: 54
```

Groovy внутри JVM

Компиляция

1. `Greeting.groovy`
   исходный код
2. `groovyc`
   компилятор Groovy
3. `Greeting.class`
   байт-код JVM
4. `JVM`
   загружает и выполняет класс

Память при выполнении

Стек потока
кадры вызовов и локальные переменные
свой у каждого потока

Куча
объекты процесса
потоки могут разделять ссылки

Сборщик мусора
освобождает недостижимые объекты

Итоги третьей части

Исключения	`catch`, `finally`, `throw`, `assert`
Ввод и вывод	файлы, консоль, кодировки и закрытие ресурсов
Потоки	`start()`, `join()`, пул и `Future`
Общее состояние	гонки, критическая секция и `synchronized`
JVM и память	байт-код, стек потока, куча и сборщик мусора

Источники

Apache Groovy 6

Документация языка

API: MissingMethodException

API: MissingPropertyException

Java SE 21

API: Thread

API: ExecutorService

API: Future

Руководство по синхронизации